

인공지능 활용 실습 교안

Chapter 1. Hugging Face 활용

Transformers 라이브러리로 배우는 자연어 처리·이미지 처리 실습

■ 이 교안의 구성

1부. Hugging Face와 Transformer 이론

2부. pipeline() 함수 실습 10종

3부. 단원 정리 및 확인 문제

목차

목차.....	2
1부. Hugging Face와 Transformer 이론.....	3
■ 1. Hugging Face란 무엇인가.....	3
■ 2. Hugging Face에서 제공되는 모델의 분야.....	4
■ 3. Hugging Face의 장점.....	4
■ 4. Attention Is All You Need.....	5
■ 5. Transformer의 인코더·디코더 구조.....	5
■ 6. Transformer 기반 모델 계열.....	6
2부. pipeline() 함수 실습.....	8
■ 1. 실습 환경 준비.....	8
■ 2. pipeline() 함수의 개념.....	9
■ 3. pipeline()에서 사용할 수 있는 대표적인 task.....	9
실습 1. 텍스트 감정 분석(Text Classification).....	11
■ 영어 문장 감정 분석.....	11
■ 한국어 문장에 영어 모델을 사용했을 때.....	12
■ 한국어 모델(kolectra) 활용.....	12
실습 2. 제로샷 분류(Zero-shot Classification).....	14
실습 3. 텍스트 생성(Text Generation).....	16
■ 한국어 입력의 한계.....	16
■ 한국어 생성 모델(KoGPT2) 활용.....	17
실습 4. 마스크 채우기(Fill-Mask).....	19
■ mask 토큰 표기 방식 주의.....	19
■ 다국어 지원 모델.....	20
실습 5. 개체명 인식(NER: Named Entity Recognition).....	22
실습 6. 질의응답(Question Answering).....	23
실습 7. 문서 요약(Summarization).....	24
실습 8. 번역(Translation).....	26
■ 한국어 ↔ 영어 번역.....	26
실습 9. 이미지 설명 생성(Image-to-Text).....	29
■ 로컬 이미지 파일 사용.....	30
실습 10. 이미지 분류(Image Classification).....	31
3부. 단원 정리 및 확인 문제.....	33
■ 단원 정리.....	33
■ 확인 문제.....	33

🎯 학습목표

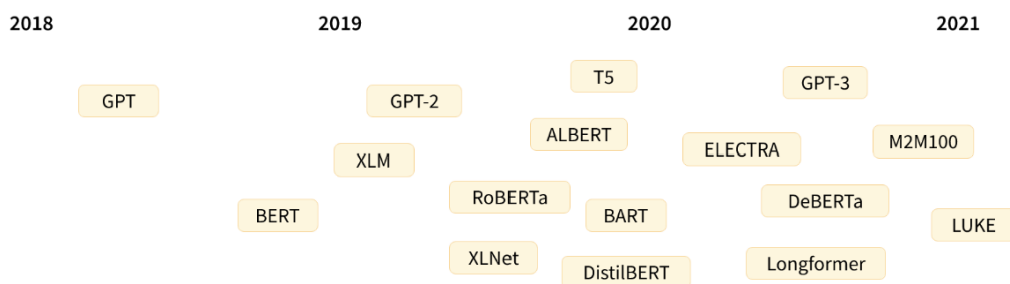
1. Hugging Face의 개념과 Transformers 라이브러리의 역할을 설명할 수 있다.
2. Transformer 모델의 등장 배경과 인코더·디코더 구조의 차이를 이해한다.
3. pipeline() 함수의 동작 원리(전처리 → 모델 → 후처리)를 설명할 수 있다.
4. pipeline() 함수를 이용하여 감정분석, 제로샷 분류, 텍스트 생성 등 10가지 자연어 처리·이미지 처리 실습을 직접 수행할 수 있다.

1 부. Hugging Face 와 Transformer 이론

■ 1. Hugging Face 란 무엇인가

Hugging Face는 딥러닝 모델과 자연어 처리 기술을 개발하고 공유하는 회사로서, 다양한 딥러닝 프레임워크를 지원하는 라이브러리와 도구를 제공하고 있다. 2016년에 설립되었으며 뉴욕에 본사를 두고 있는 머신러닝 기술 오픈소스 및 플랫폼 제공 업체이다.

Hugging Face의 대표적인 제품으로는 Transformers 라이브러리가 있다. 이 라이브러리는 자연어 처리 모델을 구현하고 학습시키는 데 사용되며, BERT, GPT, XLNet 등 다양한 모델을 제공한다. 이러한 모델들은 자연어 이해, 기계 번역, 질문 응답, 감성 분석 등 다양한 자연어 처리 태스크에서 우수한 성능을 보여주고 있다.



[그림 1] 연도별 주요 트랜스포머 계열 모델의 등장

Hugging Face는 Pretrained Models라는 모델 저장소를 제공하여 다양한 언어와 태스크에 대한 사전 학습된 모델을 제공하고 있다. 이러한 사전 학습된 모델은 전이 학습(Transfer Learning)을 통해 다양한 자연어 처리 태스크에서 우수한 성능을 내는 데 활용될 수 있다.

또한 Hugging Face는 딥러닝 모델의 학습 및 평가를 위한 도구도 함께 제공하고 있다. 예를 들어 Trainer는 딥러닝 모델의 학습을 쉽게 구현할 수 있는 인터페이스를 제공하며, Accelerate는

다중 GPU 환경에서 딥러닝 모델을 빠르게 학습할 수 있도록 지원하는 도구이다.

■ 2. Hugging Face 에서 제공되는 모델의 분야

분야	대표 작업(Task)
 자연어 처리(NLP)	텍스트 분류, 개체명 인식, 질문 응답, 언어 모델링, 요약, 번역, 텍스트 생성
 컴퓨터 비전	이미지 분류, 객체 감지, 이미지 분할
 오디오	자동 음성 인식, 오디오 분류
 멀티모달	광학 문자 인식, 문서 정보 추출, 비디오 분류, 시각적 질문 응답

■ 3. Hugging Face 의 장점

- 모델을 처음부터 개발하고 학습시킬 필요가 없다. 사전 학습된 모델을 불러와 자연어 처리 등의 작업을 바로 수행할 수 있으며, 필요에 따라 특정 작업에 맞게 미세 조정(파인튜닝)할 수도 있다.
- 간단하고 사용자 친화적인 인터페이스가 제공되어 단 몇 줄의 코드만으로 사용할 수 있다.
- 적극적인 커뮤니티가 사전 학습된 모델을 지속적으로 공유하면서 라이브러리의 성장에 기여하고 있다.
- 'Models'로 공개된 학습이 끝난 모델은 'Spaces'를 통해 간단한 데모 애플리케이션으로 제작·공개할 수도 있다.
- 음성 인식이나 분자의 3차원 구조 예측에 이르기까지 다양한 작업에 대응하며, 사전 학습된 모델의 활용으로 컴퓨팅 비용과 시간을 절감할 수 있다.
- 자연어 처리의 토큰화를 간단한 코드로 수행할 수 있는 Tokenizer 라이브러리를 제공하며, 다양한 토큰화 알고리즘을 빠르게 실행할 수 있다.

💡 출처

Hugging Face — NLP Course

『트랜스포머를 활용한 자연어 처리』(한빛미디어) 튜토리얼 번역서

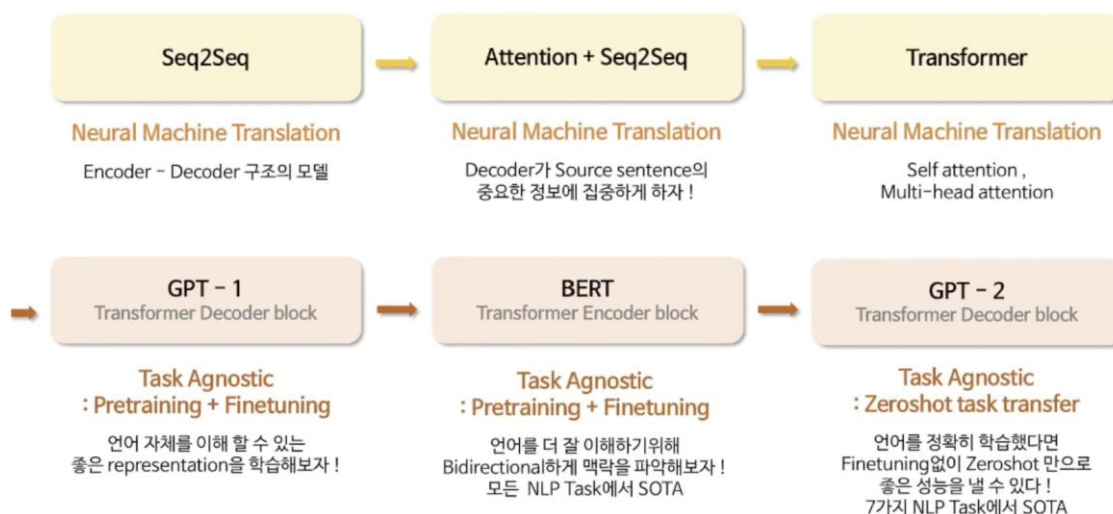
<https://huggingface.co/learn/llm-course/ko/chapter1/3?fw=pt>

■ 4. Attention Is All You Need

“Attention Is All You Need”는 구글의 연구원들이 2017년에 발표한 논문으로, 자연어 처리 분야에서 가장 혁신적인 모델 가운데 하나인 Transformer를 소개하였다. 이 논문은 Seq2Seq 모델에서 사용되는 RNN, LSTM 등이 가진 장기 의존성 문제와 병목 현상을 해결하기 위해, Self-Attention 메커니즘을 사용한 Transformer 모델을 제안하였다.

Transformer 모델은 인코더와 디코더로 구성되며, 입력 시퀀스를 인코더에서 인코딩하고 디코더에서 디코딩하여 출력 시퀀스를 생성한다. Self-Attention 메커니즘은 입력 시퀀스의 각 원소가 다른 원소들과 상호작용하도록 하는 방법으로, 긴 시퀀스에서도 장기 의존성 문제를 효과적으로 해결할 수 있다.

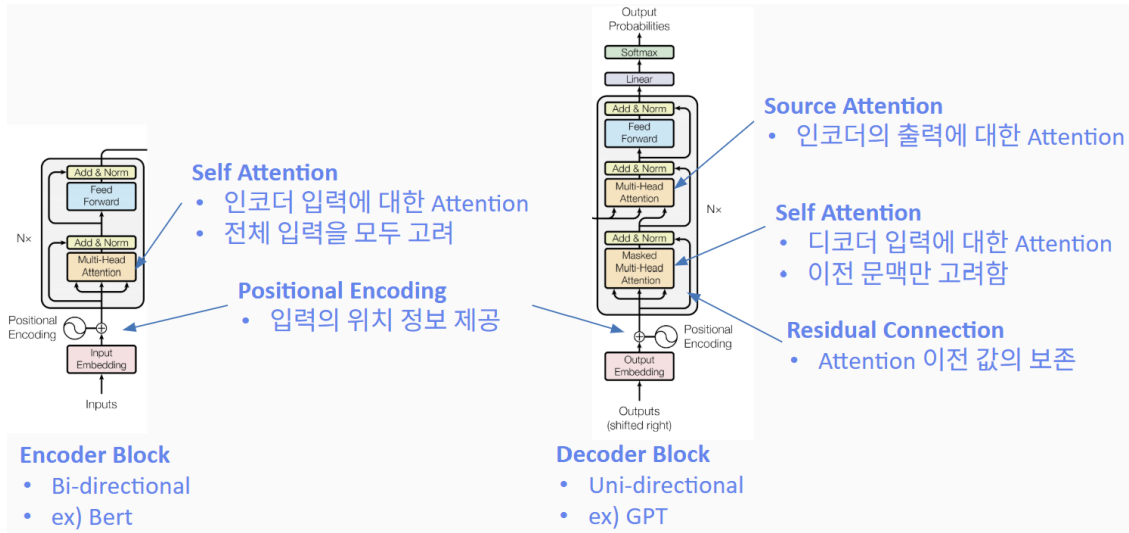
“Attention Is All You Need”는 Seq2Seq 모델에서 Transformer 모델로의 전환을 이끌었으며, 이후 자연어 처리 분야에서 매우 중요한 모델로 자리 잡았다. 이 모델은 기계 번역, 요약, 질문 응답, 대화 모델링 등에서 매우 효과적인 성능을 보여주고 있다.



[그림 2] Seq2Seq에서 Transformer로 이어지는 자연어 처리 모델의 발전 과정

■ 5. Transformer의 인코더·디코더 구조

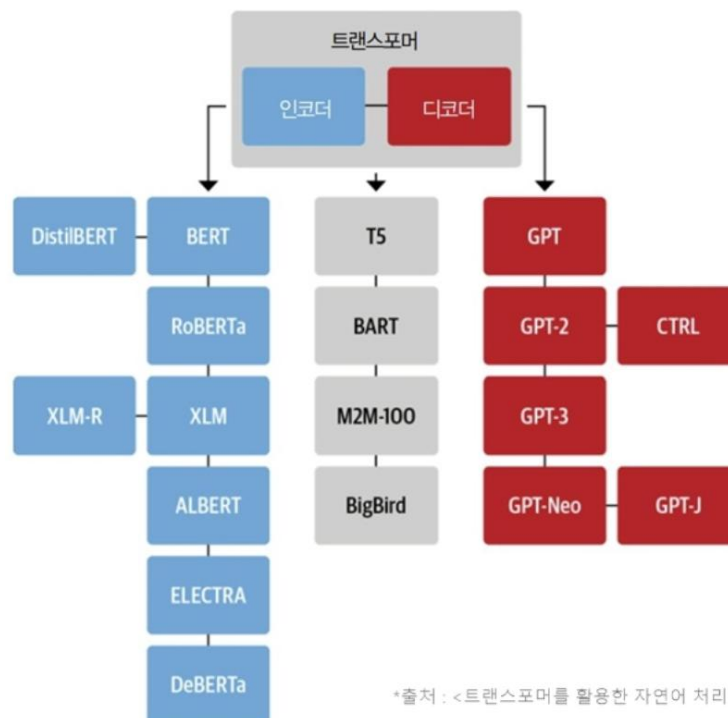
Transformer는 인코더(Encoder)와 디코더(Decoder)라는 두 개의 블록으로 이루어진다. 인코더는 입력 문장 전체를 양방향(Bi-directional)으로 고려하는 Self-Attention을 수행하며, 대표적으로 BERT가 이러한 인코더 구조를 사용한다. 반면 디코더는 이전 문맥만을 참고하는 단방향(Uni-directional) Self-Attention과, 인코더의 출력을 참고하는 Source Attention을 함께 사용하며, 대표적으로 GPT가 이러한 디코더 구조를 사용한다.



[그림 3] Transformer 인코더 블록과 디코더 블록의 구조 비교

6. Transformer 기반 모델 계열

Transformer 아키텍처는 활용 방식에 따라 크게 인코더 전용 모델, 디코더 전용 모델, 인코더-디코더 모델의 세 계열로 구분할 수 있다.



[그림 4] 트랜스포머 기반 모델의 계열 분류

계열	구조 특징	대표 모델	주요 활용 분야
인코더(Encoder-only)	양방향 Self-Attention, 문맥 이해에 강점	BERT, RoBERTa, DistilBERT, ALBERT, ELECTRA	텍스트 분류, 개체명 인식, 질문 응답 등 자연어 이해(NLU)

계열	구조 특징	대표 모델	주요 활용 분야
디코더(Decoder-only)	단방향 Self-Attention, 다음 단어 예측에 강점	GPT, GPT-2, GPT-3, GPT-Neo	텍스트 생성
인코더-디코더 (Encoder-Decoder)	입력을 인코딩 후 디코딩하여 새로운 시퀀스 생성	T5, BART, M2M-100, BigBird	번역, 요약 등 입력→출력 변환 작업

· 대표적인 인코더 모델

트랜스포머 아키텍처를 기반으로 한 첫 번째 인코더 유형의 모델은 BERT이다. 이 모델이 공개될 당시 유명한 GLUE 벤치마크(난이도가 다양한 작업으로 자연어 이해 능력을 측정하는 벤치마크)에서 기존 모델을 모두 능가하였다. 이후 성능을 더 향상시키기 위해 BERT의 사전 학습 목표와 구조가 지속적으로 조정되었으며, 인코더 유형의 모델은 여전히 연구와 산업 분야의 자연어 이해(NLU) 작업, 즉 텍스트 분류, 개체명 인식, 질문 응답 등에서 널리 활용되고 있다.

모델	특징
BERT	사전 학습 목표로 마스킹된 토큰을 예측하는 마스크드 언어 모델링(MLM)과, 한 문장이 다른 문장 뒤에 이어질 확률을 판단하는 다음 문장 예측(NSP)을 사용한다.
DistilBERT	지식 정제(Knowledge Distillation) 기법으로 BERT보다 약 40% 적은 메모리를 사용하고 약 60% 더 빠르면서도, BERT 성능의 약 97% 수준을 달성한다.
RoBERTa	더 많은 훈련 데이터로 더 큰 배치에서 더 오래 학습하고 NSP 작업을 제거하여, 원래 BERT 모델보다 성능이 크게 향상되었다.

2 부. pipeline() 함수 실습

■ 1. 실습 환경 준비

실습에 앞서 아래 명령으로 필요한 라이브러리를 설치한다. 이미 설치되어 있다면 이 과정은 생략할 수 있다.

CODE

```
# 파이토치(CPU 버전) 설치
pip install torch==2.6.0 torchvision==0.21.0 --index-url
https://download.pytorch.org/whl/cpu

# 의존 라이브러리 설치
pip install sympy==1.13.1 fsspec
pip install transformers==4.57.3
pip install sentencepiece
pip install hf_xet
pip install sacremoses
```

실습 코드에서는 불필요한 경고 메시지가 많이 출력되므로, 아래 코드를 각 노트북의 가장 앞부분에서 한 번 실행하여 경고 메시지를 정리한다.

CODE

```
import warnings
import os
import logging

# 경고 메시지 제거
warnings.filterwarnings('ignore')

# transformers 라이브러리의 로깅 레벨을 ERROR로 조정 (경고 숨김)
logging.getLogger("transformers").setLevel(logging.ERROR)

# Hugging Face 캐시 관련 symlink 경고 제거
os.environ['HF_HUB_DISABLE_SYMLINKS_WARNING'] = '1'
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
```

💡 참고

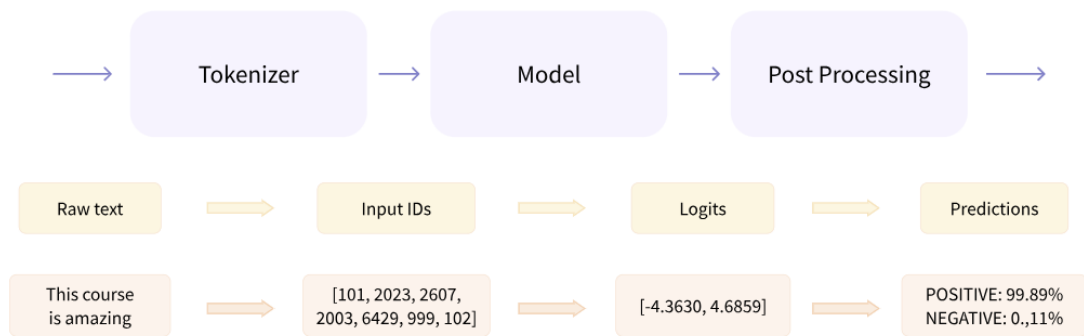
모델을 처음 사용할 때는 “사용자 폴더\w.cache\huggingface”와 같은 경로에 모델 파일이 자동으로 다운로드되므로 다소 시간이 걸릴 수 있다. 두 번째 실행부터는 캐시된 파일을 사용하므로 속도가 빨라진다.

■ 2. pipeline() 함수의 개념

Transformers 라이브러리에서 가장 기본이 되는 객체는 pipeline() 함수이다. 이 함수는 특정 모델과 그 모델의 동작에 필요한 전처리·후처리 단계를 하나로 연결해 주므로, 사용자는 텍스트(또는 이미지 등)를 직접 입력하는 것만으로 이해하기 쉬운 형태의 결과를 얻을 수 있다.

pipeline()에 입력이 전달되면 내부적으로 다음 세 단계가 순서대로 실행된다.

- 전처리(Preprocessing): 입력 데이터를 모델이 이해할 수 있는 형식(토큰 ID 등)으로 변환한다.
- 모델 추론(Model): 전처리된 입력을 모델에 전달하여 예측값(logits)을 계산한다.
- 후처리(Postprocessing): 모델의 예측값을 사람이 이해할 수 있는 형태(레이블, 확률 등)로 변환한다.



[그림 5] pipeline() 함수 내부에서 일어나는 세 단계 처리 과정

■ 3. pipeline()에서 사용할 수 있는 대표적인 task

pipeline() 함수는 task 매개변수를 통해 어떤 종류의 작업을 수행할지 지정한다. 이 교안에서 실습할 task는 다음과 같다.

task 값	설명
text-classification (별칭 sentiment-analysis)	감정 분석, 뉴스 분류, 리뷰 분류 등 문장 분류
zero-shot-classification	레이블에 대한 별도 학습 없이 후보 레이블 중에서 분류
text-generation	GPT 계열 모델을 이용한 텍스트 생성
fill-mask	문장 안의 빈칸(마스크)에 들어갈 단어 예측
ner (token-classification의 별칭)	개체명 인식(사람, 조직, 장소 등 라벨링)

task 값	설명
question-answering	주어진 지문(context)을 근거로 질문에 답변
summarization	긴 문서를 짧게 요약
translation	서로 다른 언어 간 번역
image-to-text	이미지 내용을 설명하는 문장 생성
image-classification	이미지가 어떤 대상인지 분류

실습 1. 텍스트 감정 분석(Text Classification)

텍스트 감정 분석은 문장이 긍정적인 내용인지 부정적인 내용인지를 판별하는 대표적인 문장 분류(text-classification) 작업이다. 처리 과정은 토큰화 → 워드 임베딩 → 모델 추론 → 예측(prediction)의 순서로 이루어지며, pipeline() 함수는 이 과정을 내부적으로 자동 처리해 준다.

■ 영어 문장 감정 분석

CODE

```
from transformers import pipeline

# task="text-classification" : 감정분석 전용으로 쓸 때는 "sentiment-analysis"도 동일하게 동작
classifier = pipeline(task="text-classification",
                      model="distilbert/distilbert-base-uncased-finetuned-sst-2-english")

classifier("I've been waiting for a Hugging Face course my whole life.")
```

▶ 실행 결과

```
[{'label': 'POSITIVE', 'score': 0.9982948899269104}]
```

여러 문장을 한 번에 감정분석하고 싶은 경우에는 리스트(list) 형태로 문장을 전달하면 된다.

CODE

```
classifier([
    "I've been waiting for a Hugging Face course my whole life.",
    "I hate this so much!"
])
```

▶ 실행 결과

```
[{'label': 'POSITIVE', 'score': 0.9982948899269104},
 {'label': 'NEGATIVE', 'score': 0.9994558691978455}]
```

💡 모델 용량 확인하기

AutoModel.from_pretrained()로 모델을 불러온 뒤, 파라미터 수를 세어 모델의 대략적인 메모리 용량을 계산할 수 있다.

 CODE

```
from transformers import AutoModel

model = AutoModel.from_pretrained("distilbert/distilbert-base-uncased-finetuned-sst-2-english")

# 전체 파라미터 개수 세기
total_params = sum(p.numel() for p in model.parameters())
print(f"전체 파라미터 수: {total_params:,}")
print(f"전체 파라미터 수: {total_params*4/1024/1024:.3f}MB") # 파라미터 1개 = 4byte(float32) 가정
```

 한국어 문장에 영어 모델을 사용했을 때

영어로 학습된 모델에 한국어 문장을 입력하면 다음과 같이 감정을 정확하게 판별하지 못하는 경우가 발생할 수 있다.

 CODE

```
classifier("이 물건 정말 사고 싶어요")
```

 실행 결과

```
[{'label': 'POSITIVE', 'score': 0.8577594757080078}]
```

따라서 한국어 문장의 감정을 정확하게 분석하려면 한국어로 학습된 모델을 별도로 사용해야 한다. Hugging Face 홈페이지(huggingface.co/models)에서 왼쪽 Tasks 목록의 text-classification 과 Languages 목록의 Korean을 함께 선택하면 한국어 지원 모델을 찾을 수 있다.

 한국어 모델(kolectra) 활용 CODE

```
classifier = pipeline("sentiment-analysis",
                      model="daekeun-ml/kolectra-small-v3-nsmc")

# 테스트 문장
texts = ["너가 좋아", "당신이 싫어요", "오늘 기분이 최고야", "정말 짜증나", "힘들어요"]
classifier(texts)
```

 CODE

```
# 라벨 값을 읽기 쉬운 형태로 변환하여 출력
for text, result in zip(texts, classifier(texts)):
    label = "긍정" if result['label'] == '1' else "부정"
    print(f"'{text}' → {label} ({result['score']:.2%})")
```

▶ 실행 결과

```
'너가 좋아' → 긍정 (00.00%)
'당신이 싫어요' → 부정 (00.00%)
'오늘 기분이 최고야' → 긍정 (00.00%)
'정말 짜증나' → 부정 (00.00%)
'힘들어요' → 부정 (00.00%)
```

※ 실제 점수(score)는 실행 환경에 따라 달라질 수 있으므로 직접 실행하여 확인한다.

실습 2. 제로샷 분류(Zero-shot Classification)

제로샷 분류는 기계학습 및 자연어 처리에서 개별 작업에 대한 별도의 학습(파인튜닝) 없이도 분류 작업을 수행할 수 있는 방식이다. 분류하고자 하는 후보 레이블(candidate_labels)만 지정해 주면, 모델이 사전에 학습한 언어 지식을 바탕으로 입력 문장이 어떤 레이블에 가장 가까운지 확률로 계산해 준다.

CODE

```
classifier = pipeline(task="zero-shot-classification",
                      model="facebook/bart-large-mnli")

classifier(
    "I have a problem with my iphone that needs to be resolved asap!!",
    candidate_labels=["urgent", "not urgent", "phone", "tablet", "computer"]
)
```

▶ 실행 결과

```
{'sequence': 'I have a problem with my iphone that needs to be resolved asap!!',
 'labels': ['urgent', 'phone', 'computer', 'not urgent', 'tablet'],
 'scores': [0.5036, 0.4788, 0.0126, 0.0027, 0.0023]}
```

결과를 살펴보면 "아이폰 문제를 최대한 빨리 해결해야 한다"는 문장이 urgent(긴급함)와 phone(전화기) 레이블에 대해 높은 확률을 보이는 것을 확인할 수 있다.

CODE

```
classifier(
    "This is a course about the Transformers library",
    candidate_labels=["education", "politics", "business"]
)
```

▶ 실행 결과

```
{'sequence': 'This is a course about the Transformers library',
 'labels': ['education', 'business', 'politics'],
 'scores': [0.8446, 0.1120, 0.0434]}
```

education(교육) 약 84.5%, business(사업) 약 11.2%, politics(정치) 약 4.3%
→ 이 문장은 "교육"에 관련된 내용이라고 결론지을 수 있다.

 CODE

```
sequence_to_classify = "One day I well see the world"  
candidate_labels = ['travel', 'cooking', 'dancing']  
classifier(sequence_to_classify, candidate_labels)
```

 정리

제로샷 분류는 새로운 분류 기준(레이블)이 자주 바뀌거나, 학습 데이터를 준비하기 어려운 상황에서 특히 유용하게 활용할 수 있다.

실습 3. 텍스트 생성(Text Generation)

텍스트 생성은 주어진 문장(prompt) 뒤에 이어질 내용을 모델이 예측하여 생성하는 작업으로, 주로 GPT 계열의 디코더 모델이 사용된다.

💡 모델 크기와 메모리 참고

GPT-2(파라미터 약 124M) 기준으로 파라미터만 저장하는 데도 $124 \times 1,000,000 \times 4\text{byte} \approx 496\text{MB}$ 의 메모리가 필요하다. 학습 시에는 활성화 함수 계산, 옵티마이저 상태, 기울기(미분) 저장 등으로 3~4배의 추가 메모리가 더 필요하여 결과적으로 1~2GB 수준의 메모리가 필요해질 수 있다. (정밀도를 낮추는 양자화를 적용하면 메모리를 1/4로 줄이는 것도 가능하다.)

📄 CODE

```
from transformers import pipeline

generator = pipeline("text-generation", "gpt2") # GPT-3부터는 Hugging Face에 공개
# 되어 있지 않음
result = generator("in this course. We will teach you how to",
                   pad_token_id=generator.tokenizer.eos_token_id) # GPT-2는 원래 pad_token이
# 없어 미지정 시 오류가 날 수 있음
print(result[0]['generated_text'])
```

▶ 실행 결과

```
in this course. We will teach you how to prepare in the English language in order
to become a professional journalist, and how to get yourself into the proper and
professional ranks. We will show you how to make sure you have a good relationship
with the media and how to stay on top of your journalism.
```

■ 한국어 입력의 한계

GPT-2, DistilGPT2는 영어 위주로 학습된 모델이므로 한국어 문장을 입력하면 자연스럽지 않은 결과가 생성된다.

 CODE

```
from transformers import pipeline
generator = pipeline("text-generation", "gpt2")
generator("이 과정은 다음과 같은 방법을 알려드려요.",
         pad_token_id=generator.tokenizer.eos_token_id)
```

▶ 실행 결과

이 과정은 다음과 같은 방법을 알려드려요.

같은 방법을 알려드려요. 이식의 이식의 이정은 다음과 알려드려요. ...
(문장이 반복되거나 의미가 통하지 않는 결과가 나타난다.)

■ 한국어 생성 모델(KoGPT2) 활용

한국어 텍스트를 자연스럽게 생성하려면 한국어로 학습된 모델을 사용해야 한다. `skt/kogpt2-base-v2`는 대표적인 한국어 생성 모델이다. 아래와 같이 다양한 생성 옵션을 지정하여 결과를 조절할 수 있다.

 CODE

```
from transformers import pipeline, set_seed

generator = pipeline("text-generation",
                    model="skt/kogpt2-base-v2",
                    device=-1) # CPU 사용, GPU를 사용하려면 0

result = generator("이 과정은 다음과 같은 방법을 알려드려요",
                  pad_token_id=generator.tokenizer.eos_token_id,
                  max_new_tokens=100,          # 생성할 최대 토큰 수
                  num_return_sequences=1,      # 생성할 문장 개수
                  do_sample=True,             # 샘플링 사용 (다양성 확보)
                  top_k=50,                   # 확률 상위 k개 토큰 중에서만 샘플링
                  top_p=0.95,                 # 누적 확률 95%가 될 때까지의 후보 토큰만 사용
                  temperature=1.2,            # 값이 낮을수록 보수적, 높을수록 창의적인 문장 생성
                  no_repeat_ngram_size=2      # 동일한 n-gram의 반복 방지
                  )
print(result[0]['generated_text'])
```

옵션	의미
<code>max_new_tokens</code>	새로 생성할 최대 토큰(단어 조각) 수
<code>num_return_sequences</code>	생성할 결과 문장의 개수

옵션	의미
do_sample	True로 설정하면 확률적 샘플링으로 다양한 문장을 생성
top_k / top_p	확률이 높은 후보 토큰의 범위를 제한하여 품질과 다양성을 조절
temperature	값이 낮을수록 보수적(안정적), 높을수록 창의적(무작위적)인 문장 생성
no_repeat_ngram_size	지정한 길이의 n-gram이 문장 내에서 반복되지 않도록 제한

실습 4. 마스크 채우기(Fill-Mask)

마스크 채우기는 문장 안에 마스크(mask) 토큰으로 가려진 빈칸에 들어갈 단어를 모델이 예측하는 작업이다. top_k 옵션으로 확률이 높은 순서대로 몇 개의 후보 단어를 보여줄지 지정할 수 있다.

CODE

```
unmasker = pipeline("fill-mask",
                    "distilbert/distilroberta-base")

unmasker("I'm going to hospital and meet a <mask>",
        top_k=2)
```

▶ 실행 결과

```
[{'score': 0.1928, 'token_str': ' doctor',
  'sequence': "I'm going to hospital and meet a doctor"},
 {'score': 0.0679, 'token_str': ' psychiatrist',
  'sequence': "I'm going to hospital and meet a psychiatrist"}]
```

CODE

```
unmasker("Hello I'm a <mask> model.") # top_k를 지정하지 않으면 기본값 5개가 출력됨
```

▶ 실행 결과

```
[{'score': 0.0467, 'token_str': ' business', 'sequence': "Hello I'm a business
model."},
 {'score': 0.0385, 'token_str': ' freelance', 'sequence': "Hello I'm a freelance
model."},
 {'score': 0.0331, 'token_str': ' fashion', 'sequence': "Hello I'm a fashion
model."},
 {'score': 0.0302, 'token_str': ' role', 'sequence': "Hello I'm a role model."},
 {'score': 0.021 , 'token_str': ' ...', 'sequence': "..."}]
```

■ mask 토큰 표기 방식 주의

모델마다 마스크 토큰의 표기 방식이 다르므로, 사용하는 모델에 맞는 표기를 사용해야 한다. 예를 들어 RoBERTa 계열 모델은 <mask>를, BERT 계열 모델은 [MASK]를 사용한다.

모델 계열	마스크 토큰 표기
RoBERTa 계열 (distilroberta-base 등)	<mask>
BERT 계열 (bert-base-uncased 등)	[MASK]

■ 다국어 지원 모델

bert-base-multilingual-cased와 같은 다국어 지원 모델을 사용하면 한국어 문장에도 마스크 채우기를 적용할 수 있다. 다만 다국어 모델은 각 언어 전용 모델보다 한국어 처리 성능이 낮을 수 있다는 점에 유의한다.

CODE

```
unmasker = pipeline('fill-mask',
                    model='bert-base-multilingual-cased') # 다국어 지원 모델
unmasker("안녕하세요? 나는 [MASK] 모델입니다.", top_k=3)
```

▶ 실행 결과

```
[{'score': 0.1528, 'token_str': '이', 'sequence': '안녕하세요? 나는 이 모델입니다.'},
 {'score': 0.1085, 'token_str': '여자', 'sequence': '안녕하세요? 나는 여자 모델입니다.'},
 {'score': 0.0773, 'token_str': '가수', 'sequence': '안녕하세요? 나는 가수 모델입니다.'}]
```

💡 참고 — Inference API 방식

모델을 로컬로 내려받지 않고, Hugging Face 서버에 요청을 보내 결과만 받아오는 방식도 있다. 이 방식은 Hugging Face 회원가입 후 발급받은 Access Token이 있어야 하며, huggingface_hub 라이브러리의 InferenceClient를 사용한다.

 CODE

```
import os
from dotenv import load_dotenv
from huggingface_hub import InferenceClient

load_dotenv()  # .env 파일에 저장해 둔 HF_TOKEN 환경변수를 불러옴

client = InferenceClient(
    provider="hf-inference",
    api_key=os.environ["HF_TOKEN"],
)

result = client.fill_mask(
    "Hello. I'm a [MASK] model.",
    model="google-bert/bert-base-uncased",
)
[r.sequence for r in result]
```

실습 5. 개체명 인식(NER: Named Entity Recognition)

개체명 인식은 문장 속에서 사람 이름, 조직명, 지명 등과 같은 고유한 개체를 찾아 그 종류를 라벨링하는 작업이다. `grouped_entities=True` 옵션을 사용하면 여러 토큰으로 나뉜 개체명을 하나로 묶어서 보여준다.

CODE

```
ner = pipeline("ner",
                "dbmdz/bert-large-cased-finetuned-conll03-english",
                grouped_entities=True) # 개체를 하나로 묶어서 표시할지 여부

ner("My name is Sylvain and I work at Hugging Face in Brooklyn.")
```

▶ 실행 결과

```
[{'entity_group': 'PER', 'score': 0.9982, 'word': 'Sylvain', 'start': 11, 'end': 18},
 {'entity_group': 'ORG', 'score': 0.9796, 'word': 'Hugging Face', 'start': 33, 'end': 45},
 {'entity_group': 'LOC', 'score': 0.9932, 'word': 'Brooklyn', 'start': 49, 'end': 57}]
```

라벨	의미	예시
PER	사람(Person)	Sylvain
ORG	조직(Organization)	Hugging Face
LOC	장소(Location)	Brooklyn

개체명 인식은 뉴스 기사에서 인물·기관·지명을 자동으로 추출하거나, 이력서에서 경력 정보를 자동으로 정리하는 등 정보 추출(Information Extraction) 작업에 폭넓게 활용된다.

실습 6. 질의응답(Question Answering)

질의응답은 주어진 지문(context) 안에서 질문(question)에 해당하는 답을 찾아내는 작업이다. 이 방식은 새로운 지식을 생성하는 것이 아니라, 지문 속에서 정답이 되는 부분을 찾아내는 추출형(extractive) 질의응답에 해당한다.

CODE

```
from transformers import pipeline

question_answerer = pipeline("question-answering",
                              "distilbert/distilbert-base-cased-distilled-squad")

question_answerer(
    question="Where do I work?",
    context="My name is Sylvain and I work at Hugging Face in Brooklyn."
)
```

▶ 실행 결과

```
{'score': 0.6386, 'start': 33, 'end': 45, 'answer': 'Hugging Face'}
```

결과에서 start와 end는 지문(context) 문자열 안에서 정답이 위치한 시작·끝 인덱스를 의미한다. 아래와 같이 인덱싱으로 직접 확인해 보면 답변과 일치함을 알 수 있다.

CODE

```
context = "My name is Sylvain and I work at Hugging Face in Brooklyn."
context[33:45]
```

▶ 실행 결과

```
'Hugging Face'
```

💡 정리

질의응답 pipeline은 정답이 지문 속에 실제로 포함되어 있는 경우에만 정확하게 동작한다. 지문에 없는 내용을 질문하면 엉뚱하거나 신뢰할 수 없는 답이 나올 수 있으므로, 실제 서비스에서는 지문의 범위를 신중하게 구성해야 한다.

실습 7. 문서 요약(Summarization)

문서 요약은 긴 글의 핵심 내용을 유지하면서 짧은 문장으로 압축하는 작업이다. max_length와 min_length로 요약문의 길이 범위를 지정할 수 있으며, do_sample=False로 설정하면 매 단계에서 확률이 가장 높은 단어를 선택하는 방식(그리디 방식)으로 요약문을 생성한다.

💡 환경 참고

실습 환경의 torch 버전이 2.6보다 낮으면 일부 요약 모델의 실행이 제한될 수 있으므로, 1부의 설치 안내에 따라 torch 2.6.0 버전을 사용하는 것을 권장한다.

CODE

```
summarizer = pipeline("summarization",
                      "facebook/bart-large-cnn")

long_text = """Jin, the oldest BTS member, was discharged in June 2024.
J-Hope was discharged in October. In South Korea, all able-bodied men
aged 18 to 28 are required by law to perform 18-21 months of military
service under a conscription system meant to deter aggression from
rival North Korea. The law gives special exemptions to athletes,
classical and traditional musicians, and ballet and other dancers if
they have obtained top prizes in certain competitions. K-pop stars and
other entertainers aren't subject to such privileges. However, in
2020, BTS postponed their service until age 30 after South Korea's
National Assembly revised its Military Service Act."""

summarizer(long_text, max_length=130, min_length=30, do_sample=False)
```

▶ 실행 결과

```
[{'summary_text': "In South Korea, all able-bodied men aged 18 to 28
are required by law to perform 18-21 months of military service. The
law gives special exemptions to athletes, classical and traditional
musicians, and ballet and other dancers. BTS postponed their service
until age 30 after South Korea's National Assembly revised its
Military Service Act."}]
```

매개변수	의미
max_length / min_length	생성될 요약문 토큰 수의 최대·최소 길이
do_sample	False로 설정하면 확률이 가장 높은 단어만 선택(그리디 디코딩)

실습 8. 번역(Translation)

번역은 하나의 언어로 작성된 문장을 다른 언어의 문장으로 변환하는 작업이다. Helsinki-NLP에서 공개한 opus-mt 계열 모델은 다양한 언어 쌍에 대한 번역 모델을 제공한다.

💡 참고

번역 품질을 높이고 경고 메시지를 줄이기 위해 sacremoses 라이브러리 설치를 권장한다. (pip install sacremoses) pipeline 생성 시 model을 지정하지 않으면 오류가 발생할 수 있으므로 반드시 모델명을 함께 지정한다.

📄 CODE

```
from transformers import pipeline

translator = pipeline("translation",
                      "Helsinki-NLP/opus-mt-fr-en") # 프랑스어 → 영어

translator("Ce cours est produit par Hugging Face.")
```

▶ 실행 결과

```
[{'translation_text': 'This course is produced by Hugging Face.'}]
```

■ 한국어 ↔ 영어 번역

번역 모델은 언어 쌍과 모델에 따라 성능 차이가 크므로, 방향(한→영, 영→한)에 맞는 모델을 각각 선택해서 사용하는 것이 좋다.

 CODE

```
# 한국어 → 영어
ko2en = pipeline("translation",
                  model="Helsinki-NLP/opus-mt-ko-en")

# 영어 → 한국어
en2ko = pipeline("translation",
                  model="Helsinki-NLP/opus-mt-tc-big-en-ko")

ko_sentence = "이 문장을 영어로 번역해 주세요."
en_sentence = "I enjoy learning about AI."

print("한→영:", ko2en(ko_sentence)[0]['translation_text'])
print("영→한:", en2ko(en_sentence)[0]['translation_text'])
```

▶ 실행 결과

한→영: Please translate this sentence into English.
 영→한: US 팡창식 줄. # 영→한 모델의 번역 품질이 상대적으로 낮은 경우가 있음을 보여주는 예

여러 문장을 한꺼번에 번역할 때는 리스트로 묶어서 전달하면 된다.

 CODE

```
sentences = [
    "안녕하세요. 반갑습니다.",
    "이 시스템은 정말 유용하네요.",
    "머신러닝과 딥러닝의 차이를 설명해 주세요."
]

results = ko2en(sentences)
for res in results:
    print(res['translation_text'])
```

▶ 실행 결과

Hello. Nice to meet you.
 This system is really useful.
 Explain the difference between machine and deep-down.

 생각해보기

마지막 문장의 번역 결과("deep-down")처럼, 전문 용어가 포함된 문장은 번역 모델이 부정확하게 처리할 수 있다. 번역 결과를 그대로 사용하기보다는 검수 과정을 거치는 것이 바람직하다.

실습 9. 이미지 설명 생성(Image-to-Text)

image-to-text 작업은 입력된 이미지의 내용을 설명하는 문장을 생성하는 작업으로, 시각 정보를 처리하는 Vision Transformer(ViT)와 텍스트를 생성하는 GPT 계열 모델을 결합한 구조가 많이 사용된다.

CODE

```
from transformers import pipeline

# vit : Vision Transformer 모델
imagetotext = pipeline("image-to-text",
                        "ydshieh/vit-gpt2-coco-en")
```

이미지는 로컬 파일 경로뿐 아니라 인터넷 상의 이미지 URL도 그대로 입력할 수 있다.

CODE

```
url = 'https://huggingface.co/datasets/Narsil/image_dummy/resolve/main/parrots.png'
imagetotext(url, max_new_tokens=30) # 최대 30 토큰 이내로 이미지를 설명
```

▶ 실행 결과

```
[{'generated_text': 'two birds are standing next to each other '}]
```

PIL(Python Imaging Library)을 이용하면 URL의 이미지를 직접 불러와 화면에 표시하며 결과와 비교해 볼 수 있다.

CODE

```
from PIL import Image
import requests

image = Image.open(requests.get(url, stream=True).raw)
small_image = image.resize((200, 200))
small_image
```

CODE

```
url = "http://images.cocodataset.org/val2017/000000039769.jpg"
imagetotext(url, max_new_tokens=300)
```

▶ 실행 결과

```
[{'generated_text': 'a cat laying on a blanket next to a cat laying on a bed '}]
```

■ 로컬 이미지 파일 사용

인터넷 URL 대신 실습 컴퓨터에 저장된 이미지 파일 경로를 직접 입력할 수도 있다.

📄 CODE

```
imagetotext('images/fb.jpg', max_new_tokens=300)
```

▶ 실행 결과

```
[{'generated_text': 'a variety of fruits and vegetables are displayed on a table '}]
```

💡 참고

이미지 배열을 다루는 라이브러리(OpenCV의 numpy 배열 등)로 이미지를 불러온 경우에도 결과를 시각화할 수 있으나, image-to-text·image-classification pipeline에 입력할 때는 PIL 이미지 객체 형식을 사용하는 것이 안전하다.

실습 10. 이미지 분류(Image Classification)

이미지 분류는 입력된 이미지가 미리 정의된 여러 범주(class) 중 어디에 속하는지를 예측하는 작업이다. 아래 실습에서 사용하는 google/vit-base-patch16-224 모델은 ImageNet 데이터셋으로 학습된 Vision Transformer(ViT) 모델이다.

CODE

```
from PIL import Image

image = Image.open('images/cat.jpg')
small_image = image.resize((200, 200))
small_image  # 이미지 분류 모델에는 numpy 배열이 아닌 PIL 이미지 객체를 입력해야 한다
```

CODE

```
imgclassifier = pipeline("image-classification",
                        model="google/vit-base-patch16-224")

imgclassifier(image)
```

▶ 실행 결과

```
[{'label': 'Egyptian cat', 'score': 0.8531},
 {'label': 'tabby, tabby cat', 'score': 0.0475},
 {'label': 'tiger cat', 'score': 0.0349},
 {'label': 'Persian cat', 'score': 0.0076},
 {'label': 'Siamese cat, Siamese', 'score': 0.0038}]
```

결과는 확률(score)이 높은 순서대로 최대 5개의 후보 레이블을 보여준다. 위 결과에서는 "Egyptian cat"이 약 85.3%의 확률로 가장 높게 예측되었다.

CODE

```
imgclassifier(Image.open('images/fruits.jpeg'))
```

▶ 실행 결과

```
[{'label': 'pineapple, ananas', 'score': 0.5351},
 {'label': 'banana', 'score': 0.1940},
 {'label': 'orange', 'score': 0.1532},
 {'label': 'lemon', 'score': 0.0704},
 {'label': 'strawberry', 'score': 0.0161}]
```

💡 정리

이미지 분류 모델은 학습 시 사용한 데이터셋(ImageNet 등)에 포함된 범주에 대해서만 정확하게 분류할 수 있다. 학습 데이터에 없던 새로운 범주의 이미지를 분류하려면, 해당 범주가 포함된 데이터로 모델을 다시 학습(파인튜닝)해야 한다.

3 부. 단원 정리 및 확인 문제

■ 단원 정리

- Hugging Face는 사전 학습된 딥러닝 모델과 이를 다루는 라이브러리·도구를 제공하는 플랫폼이며, 대표 라이브러리는 Transformers이다.
- Transformer는 Self-Attention 메커니즘을 기반으로 하며, 양방향 문맥을 이해하는 인코더 (BERT 계열)와 다음 단어를 예측하는 디코더(GPT 계열), 그리고 이 둘을 결합한 인코더-디코더(T5, BART 계열) 구조로 나뉜다.
- pipeline() 함수는 전처리 → 모델 추론 → 후처리의 세 단계를 자동으로 연결해 주므로, 복잡한 코드 없이도 다양한 자연어 처리·이미지 처리 작업을 손쉽게 수행할 수 있다.
- 이번 실습에서는 감정 분석, 제로샷 분류, 텍스트 생성, 마스크 채우기, 개체명 인식, 질의응답, 문서 요약, 번역, 이미지 설명 생성, 이미지 분류의 10가지 task를 pipeline() 함수로 직접 다루어 보았다.
- 영어 전용으로 학습된 모델에 한국어를 입력하면 성능이 떨어지는 경우가 많으므로, 한국어를 다룰 때는 한국어(또는 다국어) 지원 모델을 선택하는 것이 중요하다.

■ 확인 문제

다음 물음에 답하며 이번 단원의 내용을 스스로 점검해 보자.

- 1) Hugging Face의 대표적인 라이브러리 이름과 그 역할을 설명하시오.
- 2) Transformer 모델에서 인코더와 디코더의 Self-Attention 방식은 각각 어떤 차이가 있는지 설명하시오.
- 3) pipeline() 함수가 내부적으로 수행하는 세 단계 처리 과정을 순서대로 쓰시오.
- 4) 아래 코드가 어떤 task를 수행하는 코드인지 쓰고, 실행 결과로 어떤 정보가 출력될지 설명하시오.

■ CODE

```
classifier = pipeline(task="zero-shot-classification", model="facebook/bart-large-mnli")
classifier("나는 오늘 축구 경기를 보러 간다.", candidate_labels=["스포츠", "요리", "여행"])
```

- 5) 한국어 문장의 감정 분석 정확도를 높이려면 어떤 모델을 선택해야 하는지, 그 이유와 함

께 설명하시오.

— 교안 끝 —